

**Université Hassan 1er**  
**École Nationale des Sciences Appliquées de Berrechid**

Département : Génie Informatique et Mathématiques

Filière : SI et Big Data

---

**TPs : Git & UML et Unity**

---



Réalisé par :

ABARKHA Fatima Ezzahra

Encadré par :

Pr.Hrimech

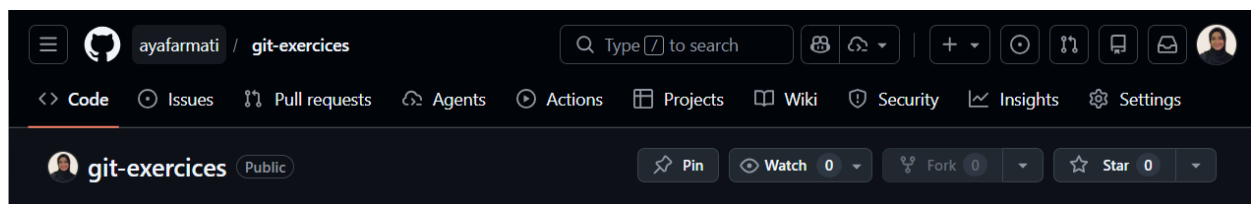
# Table des matières

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>TP0</b>                                                              | <b>2</b>  |
| <b>2</b> | <b>TP1</b>                                                              | <b>11</b> |
| <b>3</b> | <b>TP 2 : UML et Git - Approche Intégrée</b>                            | <b>14</b> |
| 3.1      | Étude de Cas 1 : Plateforme E-commerce . . . . .                        | 14        |
| 3.1.1    | Partie Modélisation UML . . . . .                                       | 14        |
| 3.1.2    | Partie Git : Workflow de développement . . . . .                        | 16        |
| 3.2      | Étude de Cas 2 : Application de Gestion Universitaire . . . . .         | 16        |
| 3.2.1    | Partie Modélisation UML . . . . .                                       | 16        |
| 3.2.2    | Partie Git : Travail Collaboratif . . . . .                             | 17        |
| 3.3      | Étude de Cas 3 : Gestion des Conflits Git . . . . .                     | 17        |
| 3.4      | Étude de Cas 4 : Projet Complet Intégré (Réservation d'Hôtel) . . . . . | 18        |
| 3.4.1    | Partie Modélisation UML . . . . .                                       | 18        |
| 3.4.2    | Partie Git : Workflow GitFlow . . . . .                                 | 19        |

# Chapitre 1

## TP0

La première étape consiste à créer le dépôt distant sur GitHub avant de le ramener sur la machine locale.



Après avoir cloné le dépôt GitHub, nous avons créé l'arborescence des dossiers (`css`, `js`, `html`) et les fichiers correspondants (`style.css`, `index.js`, `index.html`).

Une étape importante a consisté à déplacer ces dossiers à l'intérieur du répertoire `git-exercices` pour qu'ils soient pris en compte par Git. Pour vérifier que la structure est correcte et que les fichiers sont prêts à être ajoutés au suivi de version, nous utilisons la commande :

```
1 git status
```

Comme illustré dans la capture d'écran suivante, Git confirme que nous sommes sur la branche principale, qu'aucun commit n'a encore été effectué et que nos dossiers sont actuellement considérés comme des fichiers non suivis (*untracked files*).

```
MINGW64/c/Users/aya X + v
aya@DESKTOP-1FEUDBO MINGW64 ~
$ git clone https://github.com/ayafarmati/git-exercices
Cloning into 'git-exercices'...
warning: You appear to have cloned an empty repository.

aya@DESKTOP-1FEUDBO MINGW64 ~
$ mkdir css js html

aya@DESKTOP-1FEUDBO MINGW64 ~
$ touch css/style.css js/index.js html/index.html

aya@DESKTOP-1FEUDBO MINGW64 ~
$ mv css js html git-exercices/

aya@DESKTOP-1FEUDBO MINGW64 ~
$ cd git-exercices

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  css/
  html/
  js/

nothing added to commit but untracked files present (use "git add" to track)
```

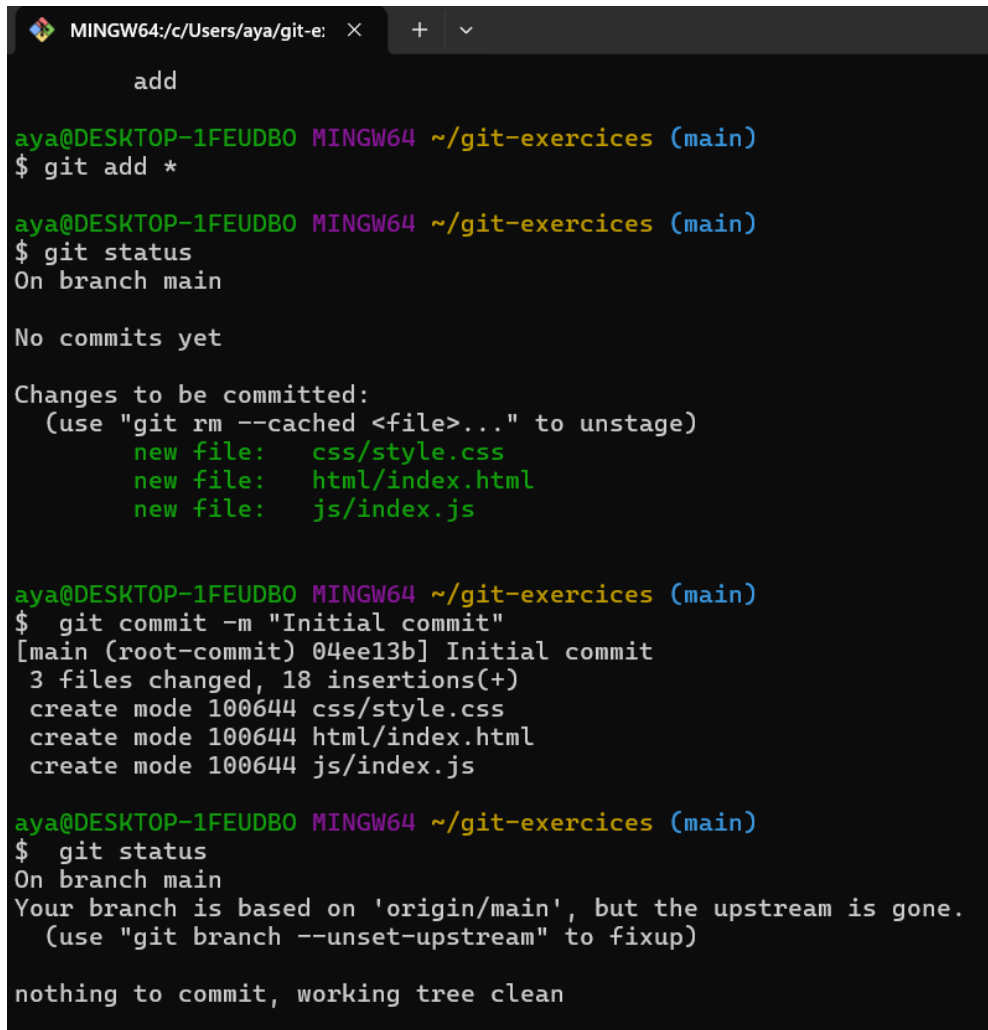
Une fois l'arborescence créée, les fichiers sont initialement "non suivis" par Git. Pour les inclure dans le prochain commit, nous devons les ajouter à l'index (staging area) avec la commande suivante :

```
1 git add *
```

En effectuant un nouveau `git status`, nous constatons que les fichiers sont passés en vert sous la mention *"Changes to be committed"*, indiquant qu'ils sont prêts à être enregistrés.

Nous procédons ensuite au premier commit du projet. Il est essentiel d'accompagner chaque commit d'un message clair pour documenter l'historique :

```
1 git commit -m "Initial commit"
```



```
MINGW64:/c/Users/aya/git-e: X + v
add

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git add *

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   css/style.css
        new file:   html/index.html
        new file:   js/index.js

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git commit -m "Initial commit"
[main (root-commit) 04ee13b] Initial commit
3 files changed, 18 insertions(+)
create mode 100644 css/style.css
create mode 100644 html/index.html
create mode 100644 js/index.js

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git status
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
  (use "git branch --unset-upstream" to fixup)

nothing to commit, working tree clean
```

Comme le montre la Figure , Git confirme la création des trois fichiers (`style.css`, `index.html`, `index.js`) avec le mode. Le message *"nothing to commit, working tree clean"* indique que toutes les modifications locales sont désormais sauvegardées dans la base de données de Git.

Pour isoler les nouveaux développements de la branche principale (`main`), nous avons créé une branche secondaire nommée `dev`. La commande `git branch` permet de lister les branches disponibles localement, où l'astérisque (\*) indique notre position actuelle.

```
1 git branch dev
2 git branch
```

Une fois la branche créée, nous basculons dessus avec la commande `checkout` pour commencer à travailler dans cet environnement séparé.

```
1 git checkout dev
```

```
aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git branch dev

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git branch
dev
* main

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git checkout dev
Switched to branch 'dev'

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (dev)
$ git status
On branch dev
nothing to commit, working tree clean
```

Comme le montre la capture d'écran, la commande `git status` confirme que nous sommes désormais sur la branche `dev` et que le répertoire de travail est propre. Cette étape est cruciale avant d'entamer des modifications sur le code source.

L'étape suivante consiste à effectuer une modification sur le fichier `style.css`. Nous allons changer la couleur de fond du bleu vers le rouge, puis enregistrer ce changement avec un message de commit structuré :

```
1 git add css/style.css
2 git commit -m "[CSS] Changed the body background color"
```

Une fois le développement terminé sur la branche `dev`, nous revenons sur la branche `main` pour fusionner le travail.

```
aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (dev)
$ git checkout -b branchName
Switched to a new branch 'branchName'

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branchName)
$ git add readme.md

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branchName)
$ git commit -m "Added a readme.md file"
[branchName 7cb7ea3] Added a readme.md file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 readme.md

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branchName)
$ git diff main dev

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branchName)
$ git branch branch-a

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branchName)
$ git branch branch-b

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branchName)
$ git diff branch-a branch-b
```

Pour explorer des scénarios plus complexes, nous créons un fichier `readme.md` à la racine du projet.

```

1 touch readme.md
2 git checkout -b branchName
3 git add readme.md
4 git commit -m "Added a readme.md file"

```

```

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (dev)
$ git checkout main
M      css/style.css
Switched to branch 'main'
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git pull
Your configuration specifies to merge with the ref 'refs/heads/main'
from the remote, but no such ref was fetched.

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git merge dev
Already up to date.

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ touch readme.md

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git status
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   css/style.css

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        readme.md

no changes added to commit (use "git add" and/or "git commit -a")

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git checkout dev
M      css/style.css
Switched to branch 'dev'

```

La commande `git diff` est essentielle pour visualiser les différences entre deux branches avant une fusion. Nous avons créé deux nouvelles branches, `branch-a` et `branch-b`, à partir de notre position actuelle.

```

1 git branch branch-a
2 git branch branch-b

```

Initialement, un `git diff branch-a branch-b` ne retourne aucun résultat car les deux branches sont identiques.

Nous avons ensuite simulé un développement parallèle en ajoutant des lignes différentes dans le fichier `readme.md` sur chaque branche :

- Sur **branch-a** : Ajout de "Branch-a instructions..."
- Sur **branch-b** : Ajout de "Branch-b instructions..."

```

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-a)
$ git branch
* branch-a
  branch-b
  branchName
  dev
  main

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-a)
$ git add readme.md

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-a)
$ git commit -m "[Readme] Added branch-a instructions"
[branch-a 1f41fe5] [Readme] Added branch-a instructions
1 file changed, 1 insertion(+)

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-a)
$ git checkout branch-b
M      css/style.css
Switched to branch 'branch-b'

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-b)
$ git add readme.md

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-b)
$ git commit -m "[Readme] Added branch-b instructions"
[branch-b ffd3b43] [Readme] Added branch-b instructions
1 file changed, 1 insertion(+)

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-b)
$ git diff branch-a branch-b
diff --git a/readme.md b/readme.md
index 51032d7..6dcd820 100644
--- a/readme.md
+++ b/readme.md
@@ -1,1 @@
-Branch-a instructions...
+Branch-b instructions...

```

Comme le montre la capture d'écran ci-dessus, `git diff` met en évidence les lignes supprimées (en rouge avec un "-") et ajoutées (en vert avec un "+") entre les deux versions du fichier `readme.md`. C'est cette divergence qui provoquera un conflit lors de la prochaine fusion.

Dans cette étape, nous créons une nouvelle branche nommée `branch-master` à partir de la branche `dev` pour simuler un environnement de production.

```

1 git checkout dev
2 git checkout -b branch-master

```

Nous fusionnons d'abord le travail provenant de `branch-a` dans notre branche simulée. Comme il n'y a pas de modifications divergentes, Git effectue un *Fast-forward*.

```

1 git merge branch-a

```

```

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git checkout dev
M      css/style.css
Switched to branch 'dev'

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (dev)
$ git checkout -b branch-master
Switched to a new branch 'branch-master'

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master)
$ git merge branch-a
Updating 489ef57..1f41fe5
Fast-forward
 readme.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 readme.md

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master)
$ git log branch-a
commit 1f41fe5dfcea74c054db1cf3a1b66281b18eb4a5 (HEAD -> branch-master, branch-a)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:52:02 2026 +0000

    [Readme] Added branch-a instructions

commit 7cb7ea3befdfe0307ceb115155fb5a9d3a1a72f2 (branchName)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:48:50 2026 +0000

    Added a readme.md file

commit 489ef573812c1f7376e5dc90f4c88a4cc0f07f8b (origin/main, main, dev)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:40:42 2026 +0000

    Initial commit

```

```

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master)
$ git log branch-master
commit 1f41fe5dfcea74c054db1cf3a1b66281b18eb4a5 (HEAD -> branch-master, branch-a)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:52:02 2026 +0000

    [Readme] Added branch-a instructions

commit 7cb7ea3befdfe0307ceb115155fb5a9d3a1a72f2 (branchName)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:48:50 2026 +0000

    Added a readme.md file

commit 489ef573812c1f7376e5dc90f4c88a4cc0f07f8b (origin/main, main, dev)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:40:42 2026 +0000

    Initial commit

```

L'historique des commits (`git log`) confirme que `branch-master` contient désormais le commit `"[Readme] Added branch-a instructions"` ainsi que l'ajout initial du fichier `readme.md`. On constate que le pointeur `HEAD` de `branch-master` est maintenant aligné sur celui de `branch-a`.



L'étape finale de ce TP a consisté à simuler et résoudre un conflit de fusion entre deux branches divergentes, **branch-a** et **branch-b**, ayant modifié la même ligne du fichier **readme.md**.

En tentant de fusionner **branch-b** dans notre branche de simulation **branch-master**, Git a interrompu l'opération en signalant un conflit de contenu (*Merge conflict*). L'état du dépôt est alors passé en mode **MERGING**, comme l'indique le prompt du terminal.

Pour finaliser la fusion, nous avons suivi la procédure suivante :

- **Analyse du statut** : La commande `git status` a permis d'identifier le fichier **readme.md** comme étant modifié des deux côtés (*both modified*).
- **Édition manuelle** : Nous avons ouvert le fichier pour supprimer les marqueurs de conflit (`<<<<`, `=====`, `>>>>`) et choisir la version finale du texte.
- **Indexation** : Après avoir sauvegardé le fichier corrigé, nous l'avons ajouté à l'index pour marquer le conflit comme résolu.

```
aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master)
$ git merge branch-b
Auto-merging readme.md
CONFLICT (content): Merge conflict in readme.md
Automatic merge failed; fix conflicts and then commit the result.

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master|MERGING)
$ git status
On branch branch-master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   readme.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   css/style.css

no changes added to commit (use "git add" and/or "git commit -a")

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master|MERGING)
$ git add readme.md

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master|MERGING)
$ git status
On branch branch-master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   readme.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   css/style.css
```

Comme illustré dans la Figure, une fois le fichier ajouté, Git confirme que tous les conflits sont résolus (*All conflicts fixed*) mais que la fusion doit encore être conclue par un commit final.

```

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master|MERGING)
$ git commit -m "making peace between branch-a and branch-b"
[branch-master 4901179] making peace between branch-a and branch-b

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master)
$ git log
commit 4901179b7e1b07e6eda2830bcc4f1db1e6c86379 (HEAD -> branch-master)
Merge: 1f41fe5 ffd3b43
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 21:37:15 2026 +0000

    making peace between branch-a and branch-b

commit ffd3b4325e9f17fe2ce2131b1b4d57385dc504d7 (branch-b)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:53:14 2026 +0000

    [Readme] Added branch-b instructions

commit 1f41fe5dfcea74c054db1cf3a1b66281b18eb4a5 (branch-a)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:52:02 2026 +0000

    [Readme] Added branch-a instructions

commit 7cb7ea3befdfe0307ceb115155fb5a9d3a1a72f2 (branchName)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:48:50 2026 +0000

    Added a readme.md file

commit 489ef573812c1f7376e5dc90f4c88a4cc0f07f8b (origin/main, main, dev)
Author: AYA <farmati.ensa@uhp.ac.ma>
Date:   Fri Feb 20 06:40:42 2026 +0000

    Initial commit

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master)
$ |

```

L'historique final, visualisé avec la commande `git log`, confirme la convergence réussie des branches et la validation du commit de résolution de conflit intitulé.

Une fois le conflit résolu sur la branche `branch-master`, nous avons poussé l'ensemble des branches vers le dépôt distant GitHub avant de réaliser la fusion finale sur la branche principale `main`.

```

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master)
$ git push --all
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (12/12), 1.00 KiB | 343.00 KiB/s, done.
Total 12 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 1 local object.
To https://github.com/ayafarmati/git-exercices.git
* [new branch]      branch-a -> branch-a
* [new branch]      branch-b -> branch-b
* [new branch]      branch-master -> branch-master
* [new branch]      branchName -> branchName
* [new branch]      dev -> dev

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (branch-master)
$ git checkout main
M      css/style.css
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git merge branch-master
Updating 489ef57..4901179
Fast-forward
 readme.md | 5 +++++
 1 file changed, 5 insertions(+)
 create mode 100644 readme.md

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git log master
fatal: ambiguous argument 'master': unknown revision or path not in the work
ing tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

```

Comme illustré ci-dessus, la commande `git push -all` a permis de synchroniser tout notre travail local avec le serveur. La fusion finale dans la branche `main` s'est effectuée sans conflit, confirmant l'intégrité de notre historique.

# Chapitre 2

## TP1

### Exercice : Manipulation des branches

#### Exercice

1. Dans le dépôt que vous avez créé au premier exercice, créez une branche nommée `style`, et placez-vous dans cette branche.
2. Modifiez la feuille de style (par exemple pour changer la couleur de fond) et commitez vos changements.
3. Revenez sur la branche `master`. Constatez que vos changements de style ont disparu (pour l'instant).
4. Dans la branche `master`, modifiez ou ajoutez du contenu au fichier HTML, et commitez vos modifications.
5. Revenez sur la branche `style`. Constatez que vos changements de style ont réapparu, mais que vos dernières modifications dans le fichier HTML ont, elles, disparu.
6. Modifiez à nouveau la feuille de style (par exemple pour changer la police) et commitez vos changements.

FIGURE 2.1 – Exercice1 TP1

Dans cet exercice, nous avons expérimenté l'isolation des modifications entre les branches `main` et `style`.

1. **Création et premier changement** : Nous avons créé la branche `style` et modifié la couleur de fond dans le fichier CSS.

```
$ git checkout -b style
$ git add *
$ git commit -m "Changement de la couleur de fond"
```

2. **Basculement et isolation** : En retournant sur la branche `main`, nous avons constaté que les modifications du fichier CSS ne sont pas présentes, prouvant l'indépendance des environnements de travail.

3. **Nouvelle modification sur la branche style** : Après être revenus sur la branche `style`, nous avons modifié la police d'écriture. Une erreur `fatal: pathspec` est survenue car le fichier se trouvait dans un sous-dossier (`css/style.css`). Le conflit a été résolu en spécifiant le bon chemin d'accès.

```
$ git add css/style.css
$ git commit -m "Modif de la police sur branche style"
```

```
aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git checkout -b style
Switched to a new branch 'style'

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (style)
$ git add *

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (style)
$ git commit -m "Changement de la couleur de fond "
[style bbb4197] Changement de la couleur de fond
1 file changed, 1 insertion(+), 1 deletion(-)

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (style)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git checkout style
M       html/index.html
Switched to branch 'style'

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (style)
$ git add style.css
fatal: pathspec 'style.css' did not match any files

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (style)
$ git add css/style.css

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (style)
$ git commit -m "Modif de la police sur branche style"
[style f20d55b] Modif de la police sur branche style
1 file changed, 3 insertions(+), 2 deletions(-)
```

## Exercice 2 : Fusion des branches

### Exercice

1. Nous allons maintenant fusionner la branche `style` (créée à l'exercice précédent) avec la branche `master`.
2. Placez-vous dans la branche `master`, et appliquez la méthode de votre choix (ligne de commande ou interface graphique) pour y fusionner la branche `style`.
3. Constatez que toutes vos modifications (contenu HTML et style) sont maintenant visibles.
4. Constatez également (dans l'historique) que le commit ainsi créé a *deux* commit parents.

FIGURE 2.2 – Exercice2 TP1

Nous avons procédé à l'intégration des travaux de design dans la branche principale :

```
$ git checkout main
$ git merge style
```

Le résultat est une unification des fichiers `index.html` et `style.css`. L'historique montre désormais un commit de fusion avec deux parents distincts.

```
aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (style)
$ git checkout main
M      html/index.html
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ git merge style
Updating 4901179..f20d55b
Fast-forward
 css/style.css | 5 +++--
 1 file changed, 3 insertions(+), 2 deletions(-)

aya@DESKTOP-1FEUDBO MINGW64 ~/git-exercices (main)
$ |
```

## Chapitre 3

# TP 2 : UML et Git - Approche Intégrée

Dans ce TP, nous allons mettre en pratique la modélisation de systèmes avec les diagrammes UML, combinée à une gestion de versions rigoureuse via Git pour un travail collaboratif simulé.

### 3.1 Étude de Cas 1 : Plateforme E-commerce

Cette étude de cas vise à concevoir une plateforme permettant la consultation de produits, l'ajout au panier et le paiement en ligne.

#### 3.1.1 Partie Modélisation UML

##### 1. Diagramme de Cas d'Utilisation

Le diagramme suivant illustre les interactions principales entre les acteurs (Client, Administrateur) et le système.

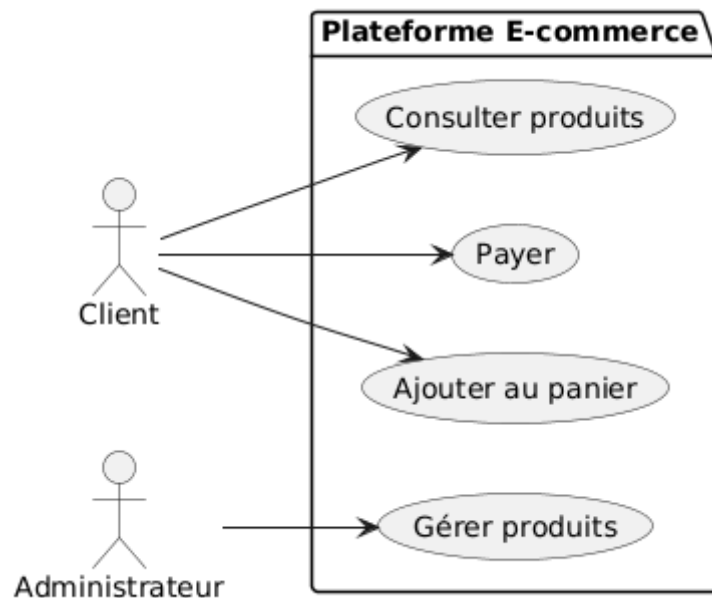


FIGURE 3.1 – Diagramme de Cas d'Utilisation - E-commerce

##### 2. Diagramme de Classes

Voici la structure statique du système montrant les entités Client, Produit, Panier, Commande et Paiement.

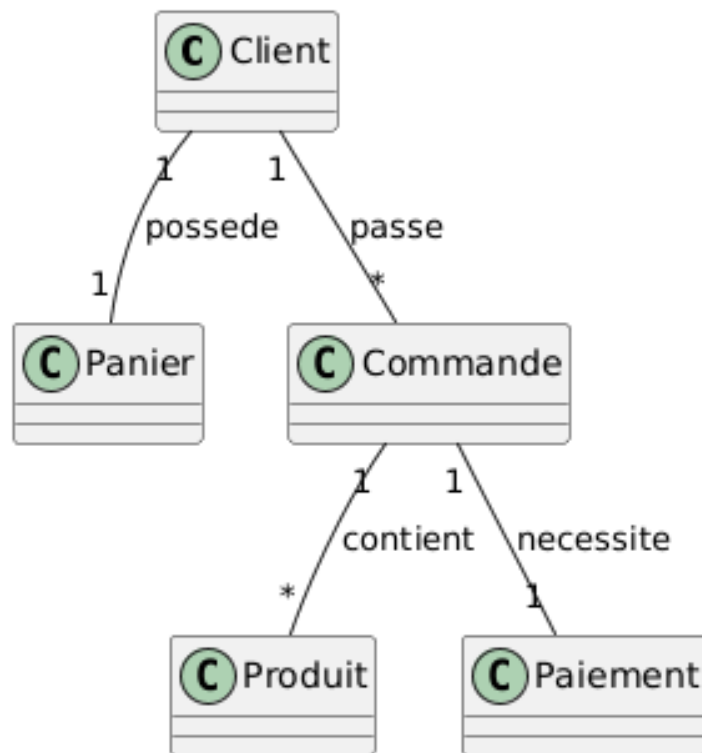


FIGURE 3.2 – Diagramme de Classes - E-commerce

### 3. Diagramme de Séquence

Ce diagramme détaille le flux d'exécution pour le scénario de paiement.

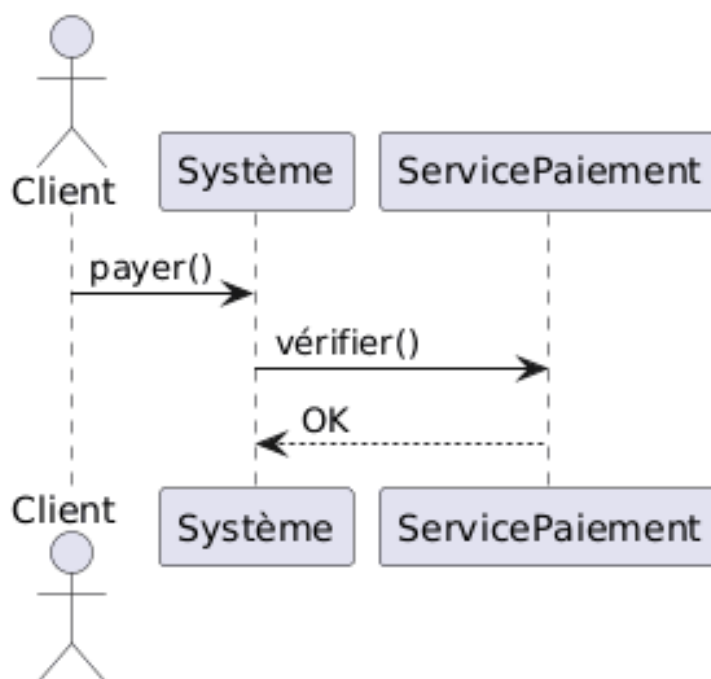


FIGURE 3.3 – Diagramme de Séquence (Paiement) - E-commerce



### 3.1.2 Partie Git : Workflow de développement

Pour développer la fonctionnalité de paiement, nous isolons notre travail dans une branche spécifique avant de la fusionner dans la branche d'intégration (develop).

```
1 # Cr ation et basculement sur la branche de la fonctionnalit
2 git checkout -b feature/paiement
3
4 # Ajout des fichiers modifi s et cr ation du commit
5 git add .
6 git commit -m "Ajout de la fonctionnalit de paiement"
7
8 # Envoi de la branche vers le d p t distant
9 git push origin feature/paiement
10
11 # Retour sur la branche d'int gration pour fusionner le travail
12 git checkout develop
13 git merge feature/paiement
```

## 3.2 Étude de Cas 2 : Application de Gestion Universitaire

Nous modélisons ici un système de gestion pour les étudiants, les enseignants et les cours.

### 3.2.1 Partie Modélisation UML

#### 1. Diagramme de Classes

Ce diagramme met en évidence l'héritage (Personne → Étudiant/Enseignant) et les associations avec les cours.

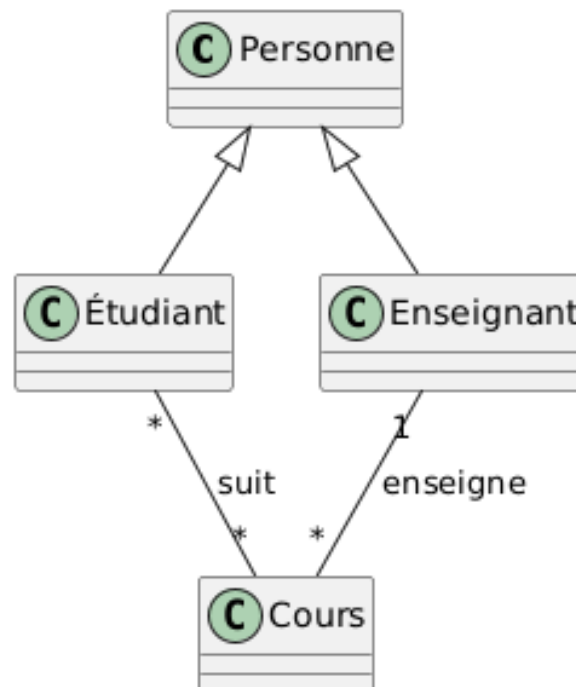


FIGURE 3.4 – Diagramme de Classes - Gestion Universitaire

## 2. Diagramme de Séquence

Illustration du processus d'inscription d'un étudiant.

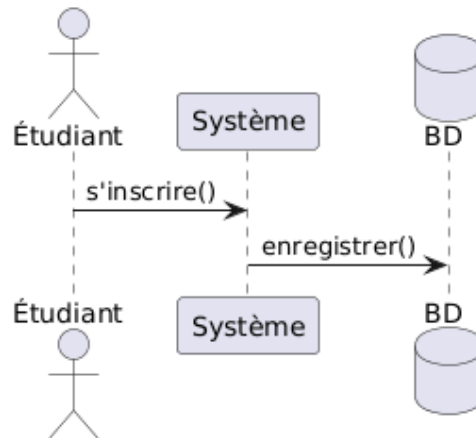


FIGURE 3.5 – Diagramme de Séquence (Inscription) - Gestion Universitaire

### 3.2.2 Partie Git : Travail Collaboratif

Dans ce scénario, chaque développeur travaille sur sa propre branche et prépare le terrain pour des *Pull Requests*.

```
1 # R cup ration des derni res mises      jour de l' quipe
2 git pull origin main
3
4 # Cr ation d'une branche pour la t che d'inscription
5 git checkout -b dev-inscription
6
7 # D veloppement en cours...
8 git add .
9 git commit -m "Impl mentation du module d'inscription"
10
11 # Pousser la branche pour cr er une Pull Request sur GitHub
12 git push origin dev-inscription
```

## 3.3 Étude de Cas 3 : Gestion des Conflits Git

Pour simuler un environnement réel, nous avons provoqué un conflit en modifiant le même fichier simultanément sur deux branches différentes.

```
1 # Simulation : Le d veloppeur 1 modifie un fichier
2 git checkout -b dev1
3 echo "Code du dev 1" > fichier.txt
4 git add fichier.txt
5 git commit -m "Modif dev1"
6
7 # Simulation : Le d veloppeur 2 modifie le m me fichier      la m me ligne
8 git checkout main
9 git checkout -b dev2
10 echo "Code du dev 2" > fichier.txt
11 git add fichier.txt
```

```

12 git commit -m "Modif_dev2"
13
14 # Tentative de fusion provoquant un conflit
15 git checkout main
16 git merge dev1
17 git merge dev2 # CONFLIT ICI
18
19 # R solution manuelle des conflits dans fichier.txt (<<<<<< ===== >>>>>>)
20 git add fichier.txt
21 git commit -m "R solution du conflit entre dev1 et dev2"

```

## 3.4 Étude de Cas 4 : Projet Complet Intégré (Réservation d'Hôtel)

Ce projet intègre l'ensemble du cycle : modélisation complète et adoption d'un flux de travail professionnel (GitFlow).

### 3.4.1 Partie Modélisation UML

#### 1. Diagramme de Cas d'Utilisation

Représentation des cas : Réserver chambre, Payer, Annuler.

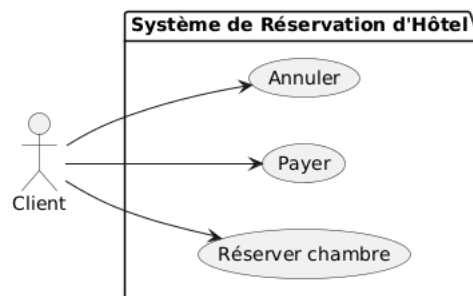


FIGURE 3.6 – Diagramme de Cas d'Utilisation - Réservation

#### 2. Diagramme de Classes

Structure liant le Client, la Réservation et la Chambre.

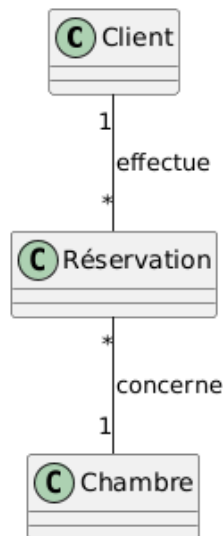


FIGURE 3.7 – Diagramme de Classes - Réservation

### 3. Diagramme de Séquence

Processus de vérification et de validation de la réservation.

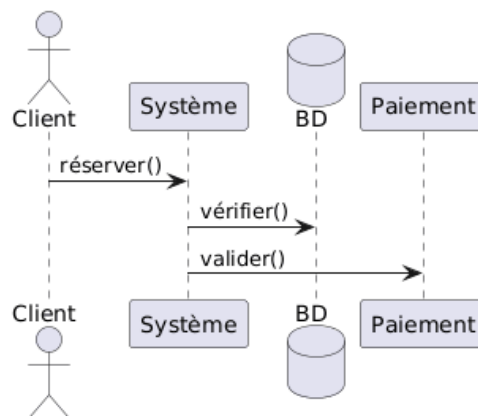


FIGURE 3.8 – Diagramme de Séquence (Réservation) - Hôtel

#### 3.4.2 Partie Git : Workflow GitFlow

Utilisation d'un flux de travail standardisé d'entreprise :

```

1  # Initialisation du projet et création de la branche de développement
2  git checkout -b develop main
3
4  # Démarrage d'une nouvelle fonctionnalité
5  git checkout -b feature/reservation-chambre develop
6
7  # Après les commits de développement...
8  git add .
9  git commit -m "Finalisation du processus de réservation"
10
11 # Intégration de la fonctionnalité dans develop
12 git checkout develop
13 git merge feature/reservation-chambre
  
```

```
14 |
15 | # Préparation d'une release pour la production
16 | git checkout -b release/v1.0 develop
17 | git checkout main
18 | git merge release/v1.0
19 | git tag -a v1.0 -m "Version 1.0 - Réservation d'hôtel"
```

# TP Unity 3D

## Création d'un Système Solaire

# Table des matières

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Introduction</b>                           | <b>2</b>  |
| <b>1 Création de la Scène</b>                 | <b>3</b>  |
| 1.1 Création du Soleil . . . . .              | 3         |
| 1.1.1 Paramètres du Soleil . . . . .          | 3         |
| 1.1.2 Application de la Texture . . . . .     | 3         |
| 1.2 Ajout des Planètes . . . . .              | 4         |
| 1.3 Configuration du Skybox . . . . .         | 4         |
| <b>2 Animation et Scripts</b>                 | <b>6</b>  |
| 2.1 Principe d'Animation . . . . .            | 6         |
| 2.2 Script d'Animation des Planètes . . . . . | 6         |
| <b>3 Contrôle de la Caméra</b>                | <b>8</b>  |
| 3.1 Script MouseAimCamera . . . . .           | 8         |
| 3.2 Paramètres Importants . . . . .           | 9         |
| <b>Conclusion</b>                             | <b>11</b> |

# Introduction

L'objectif de ce TP sous Unity 3D est la création d'une scène animée montrant un système solaire. À travers ce TP, nous allons acquérir les compétences suivantes :

- Maîtriser les rotations et les translations des objets
- Utilisation des Skybox
- Utilisation des Textures
- Création de scripts C# pour l'animation
- Contrôle de caméra



# Chapitre 1

## Création de la Scène

### 1.1 Création du Soleil

#### 1.1.1 Paramètres du Soleil

1. Insertion d'une sphère dans la scène
2. Renommage en "**Soleil**"
3. Ajustement de la taille
4. Positionnement à l'origine (0,0,0)

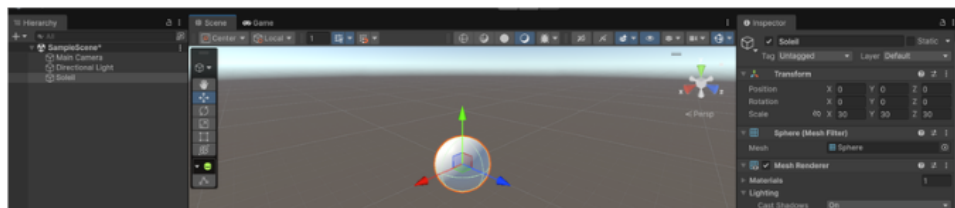


FIGURE 1.1 – Le Soleil dans la scène

#### 1.1.2 Application de la Texture

Application de la texture du soleil sur la sphère via le Material.

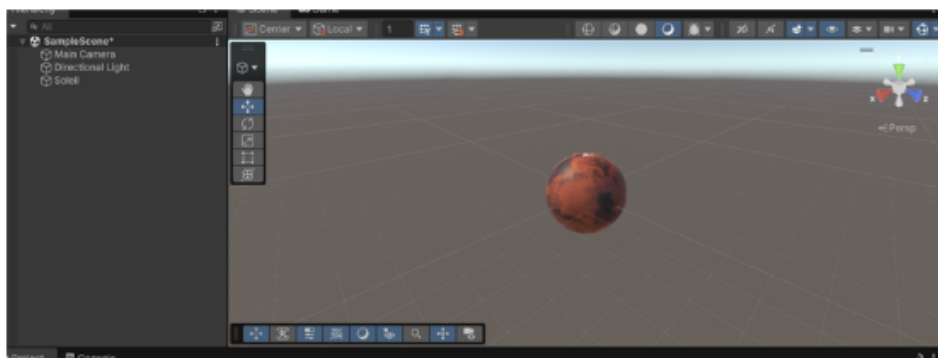


FIGURE 1.2 – Texture appliquée au Soleil

## 1.2 Ajout des Planètes

Création des planètes dans l'ordre suivant :

1. Mercure
2. Vénus
3. Terre
4. Mars
5. Jupiter
6. Saturne
7. Uranus
8. Neptune

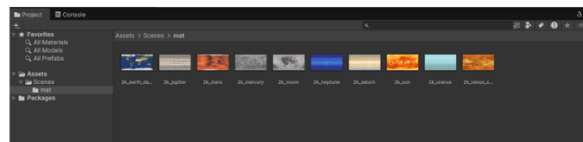


FIGURE 1.3 – Textures ppliquées aux planètes

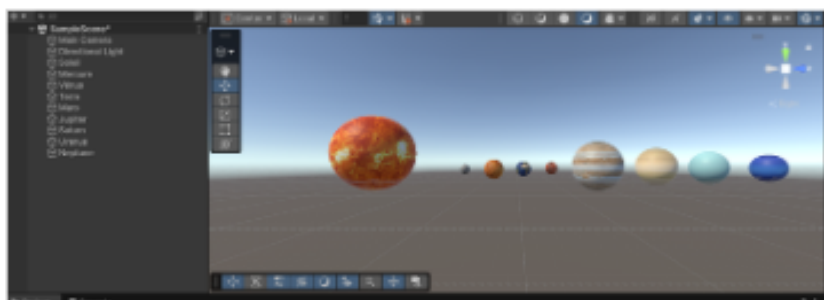


FIGURE 1.4 – Les planètes du système solaire

## 1.3 Configuration du Skybox

Ajout du **Starfield Skybox** disponible dans l'asset importé pour créer l'arrière-plan spatial.



FIGURE 1.5 – Skybox spatial appliqué

# Chapitre 2

## Animation et Scripts

### 2.1 Principe d'Animation

Chaque planète doit :

- Tourner sur elle-même (rotation)
- Tourner autour du Soleil (translation/revolution)

### 2.2 Script d'Animation des Planètes

Voici le script C# utilisé pour animer les planètes :

```
1 using UnityEngine;
2 using System.Collections;
3 using System.Collections.Generic;
4 using UnityEngine;
5
6 public class PlaneteRotation : MonoBehaviour
7 {
8     public float vitesseRotation = 35f;
9     public float vitesseOrbite = 70f;
10
11     private GameObject soleil;
12
13     void Start()
14     {
15         soleil = GameObject.Find("Soleil");
16     }
17
18     void Update()
19     {
20         // Rotation sur elle-m me
21         transform.Rotate(0, vitesseRotation * Time.deltaTime, 0);
22
23         // Rotation autour du Soleil
24         if (soleil != null)
25         {
26             transform.RotateAround(soleil.transform.position, -soleil.
27                 transform.up, vitesseOrbite * Time.deltaTime);
28         }
29     }
30 }
```

29 }

## Listing 2.1 – Script d’animation des planètes

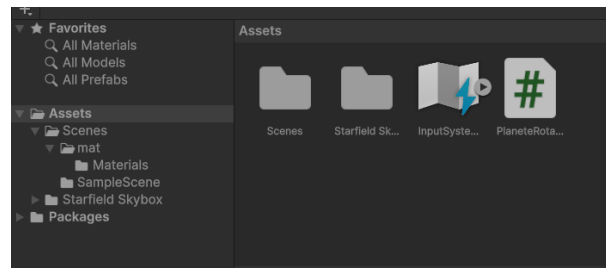


FIGURE 2.1 – Application du script d’animation

# Chapitre 3

## Contrôle de la Caméra

### 3.1 Script MouseAimCamera

Pour permettre à l'utilisateur de contrôler la caméra avec la souris, nous avons implémenté le script suivant :

```
1 using UnityEngine;
2
3 using System.Collections;
4 using System.Collections.Generic;
5 using UnityEngine;
6
7 public class MouseAimCamera : MonoBehaviour
8 {
9     public GameObject target;
10    public float rotateSpeed = 5;
11    Vector3 offset;
12
13    void Start()
14    {
15        offset = target.transform.position - transform.position;
16    }
17
18    void LateUpdate()
19    {
20        // Axe X (horizontal) - fait pivoter la cible
21        float horizontal = Input.GetAxis("Mouse X") * rotateSpeed;
22        target.transform.Rotate(0, horizontal, 0);
23
24        // Axe Y (vertical) - fait monter/descendre la cam ra
25        float vertical = Input.GetAxis("Mouse Y") * rotateSpeed;
26        target.transform.Rotate(vertical, 0, 0);
27
28        // Recalculer la position de la cam ra
29        float desiredAngle = target.transform.eulerAngles.y;
30        float desiredAngleX = target.transform.eulerAngles.x;
31
32        Quaternion rotation = Quaternion.Euler(desiredAngleX,
33        desiredAngle, 0);
34        transform.position = target.transform.position - (rotation *
35        offset);
36
37        transform.LookAt(target.transform);
38    }
39 }
```

```
36     }  
37 }
```

Listing 3.1 – Script de contrôle de caméra

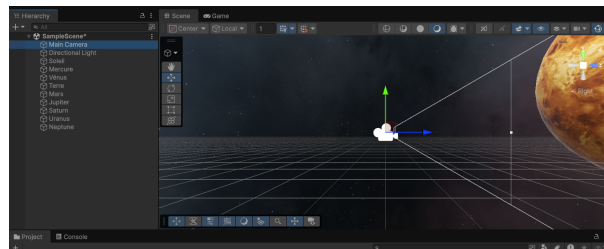


FIGURE 3.1 – Configuration du contrôle de caméra

## 3.2 Paramètres Importants

- **Target** : Doit être réglé sur **Main Camera**
- **Rotate Speed** : Vitesse de rotation (par défaut 5)
- Modification pour contrôler l'orientation selon les axes X et Y

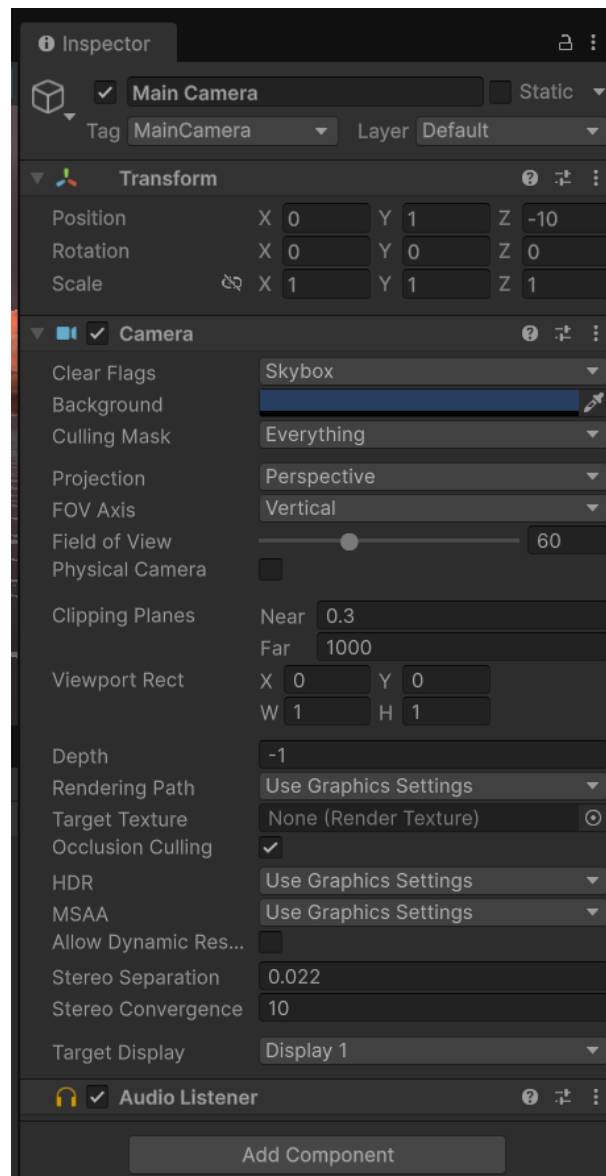


FIGURE 3.2 – Paramètres du script dans l'Inspector



# Conclusion

À travers ce TP, nous avons pu :

- **Maîtriser les transformations 3D** : Rotation et translation des objets dans Unity
- **Utiliser les textures** : Application de matériaux réalistes sur les planètes
- **Configurer un Skybox** : Création d'un environnement spatial immersif
- **Développer en C#** : Création de scripts pour l'animation automatique
- **Contrôler la caméra** : Interaction utilisateur via la souris

Ce projet nous a permis de comprendre les concepts fondamentaux de la programmation dans Unity 3D et de créer une scène interactive complète représentant notre système solaire.